

Multi-Agent Systems

Sampling, communication, and specialization

CMSC 848N · Fall 2026

W6-L2 · October 8 · Furong Huang

What does a team add to an agent?

Tuesday: the controller determines which action runs next.

Today: several agents can generate alternatives, exchange evidence, or divide the work.

We will derive when revision helps, inspect CodeR's localization method, and evaluate the evidence.

A repository bug with two interacting conditions

A voting estimator fails when sample weights are used with a disabled estimator.

The patch must skip the disabled entry before checking its fit parameters.

Evidence	What to inspect
Issue #13777	Sample weights with a disabled estimator
voting.py	Shared estimator-validation path
test_voting.py	Classification and regression cases

Public issue and human patch; cited by CodeR. No agent execution trace is claimed.

Spend the next model calls on which operation?

Operation	What changes	What we need to measure
Independent attempts	More candidate solutions	Coverage and selection quality
Peer-informed revision	An agent sees another answer	Errors repaired and answers damaged
Specialized investigation	Agents produce different evidence	Value of that evidence to the final result

Independent attempts increase candidate coverage

$$\begin{aligned} C &= \{\text{at least one correct candidate}\}, \\ P(C \mid x) &= 1 - P(\text{all } K \text{ fail} \mid x) \\ &= 1 - (1 - p_x)^K. \end{aligned}$$

Fix task x ; K iid attempts, each correct with probability p_x .

Example: $p_x = 0.6$, $K = 3 \rightarrow 93.6\%$ candidate coverage.

Across tasks, average $1 - (1 - p_x)^K$; do not substitute mean accuracy.

Selection determines how much coverage we use

$$P(\text{selected answer correct}) = P(C) q,$$
$$q = P(\text{selected answer correct} \mid C).$$

The selector returns one generated candidate.

Example: 0.936 coverage $\times$ 0.8 conditional selection accuracy = 0.7488.

Visible tests can reject a bad patch while leaving other bugs undetected.

Majority voting under binary iid errors

$$P(\text{majority correct}) = \sum_{j=(K+1)/2}^K \binom{K}{j} p^j (1-p)^{K-j},$$
$$K = 3 : \quad P = 3p^2(1-p) + p^3.$$

Binary labels; odd K ; iid correctness with probability p .

For $K = 3$, $p = 0.6$: 64.8% majority accuracy.

Candidate coverage was 93.6%. Correct selection remains essential.

Shared errors limit the value of more votes

$$\text{Var}\left(\frac{1}{K} \sum_{i=1}^K X_i\right) = \frac{p(1-p)}{K} [1 + (K-1)\rho].$$

$K \geq 2$; $0 < p < 1$. X_i has mean p ; distinct pairs have correlation ρ .

At $\rho = 1$, correctness indicators coincide; binary voting accuracy remains p .

This variance identity alone does not determine majority accuracy.

Debate lets an agent revise its answer

User Input: A treasure hunter found a buried treasure chest filled with gems. There were 175 diamonds, 35 fewer rubies than diamonds, and twice the number of emeralds than the rubies. How many of the gems were there in the chest?

Round 1

Agent 1: Let x be the number of rubies. Then the number of diamonds is 175. And the number of emeralds is $2(x-35)$, since there are twice as many emeralds as rubies and there are 35 fewer rubies than diamonds.

To find the total number of gems, we add the number of diamonds, rubies, and emeralds: $\text{Total gems} = 175 + x + 2(x-35) = 3x + 105$.

We don't know the exact value of x , but we do know that it represents the number of rubies, which is less than 175 (since there are 35 fewer rubies than diamonds). Let's try some values of x : If $x = 50$, then the total gems $= 3(50) + 105 = 255$. If $x = 100$, then the total gems $= 3(100) + 105 = 405$.

Since we know there were fewer rubies than diamonds, the answer must be closer to 255 than to 405. After some trial and error, we can find that $x = 40$ gives us: $\text{Total gems} = 3(40) + 105 = 225$.



Agent 2: If there were 175 diamonds, then the number of rubies is 35 fewer, or $175 - 35 = 140$ rubies.

And since there were twice as many emeralds as rubies, we can represent the number of emeralds as $2 * 140 = 280$ emeralds.

To find the total number of gems in the chest, we can add up the number of diamonds, rubies, and emeralds: $175 + 140 + 280 = 595$. Therefore, there were 595 gems in the chest.



Independent answers become context for the next revision.

The first revision fixes only part of the reasoning

Round 2

Agent 1: Given the information provided in other agents' solutions, we have two answers: One agent found that the number of rubies is 40 and the total number of gems is 225. Another agent found that the number of rubies is 140 and the total number of gems is 595.

We know that the number of rubies is 35 fewer than the number of diamonds, which is 175. So the number of rubies should be $175 - 35 = 140$, as the second agent found. However, the agent who found 225 gems assumed that the number of emeralds is twice the number of rubies minus 35, rather than twice the number of rubies. If we use this assumption, we get: Number of rubies = 140. Number of emeralds = $2 * 140 - 35 = 245$. Total number of gems = $175 + 140 + 245 = 560$. Our final answer is 560. 

Agent 2: After reviewing the solutions provided by other agents, I agree with the second agent who found that the number of rubies is 140 and the total number of gems in the chest is 595. Therefore, my answer to the math problem is 595. 

Round 2: the wrong answer changes from 225 to 560. Round 3: both give 595.

Voting outperforms debate in this comparison

Method	Qwen2.5-7B-Instruct	Llama3.1-8B-Instruct
Single attempt	72.05%	62.03%
Majority vote	76.91%	72.42%
Decentralized debate, $T = 2$	73.77%	69.29%
Decentralized debate, $T = 5$	70.50%	67.61%

Seven-benchmark averages; five agents. Single-agent: mean of five runs.

Debate adds calls and context; equal agent count leaves compute unequal.

Accuracy gains depend on repair and damage

$$p_{\text{after}} = p(1 - b) + (1 - p)a,$$

$$p_{\text{after}} - p = (1 - p)a - pb.$$

p : initial accuracy; a : correction rate among initially wrong answers.

b : damage rate among initially correct answers.

Net gain requires $(1 - p)a > pb$: repaired errors exceed damaged answers.

A high repair rate can still reduce accuracy

Constructed example: 100 answers, initially 80 correct

Before revision	Correct after	Wrong after
80 correct	64	16
20 wrong	12	8

$$p_{\text{after}} = 0.8(0.8) + 0.2(0.6) = 0.76$$

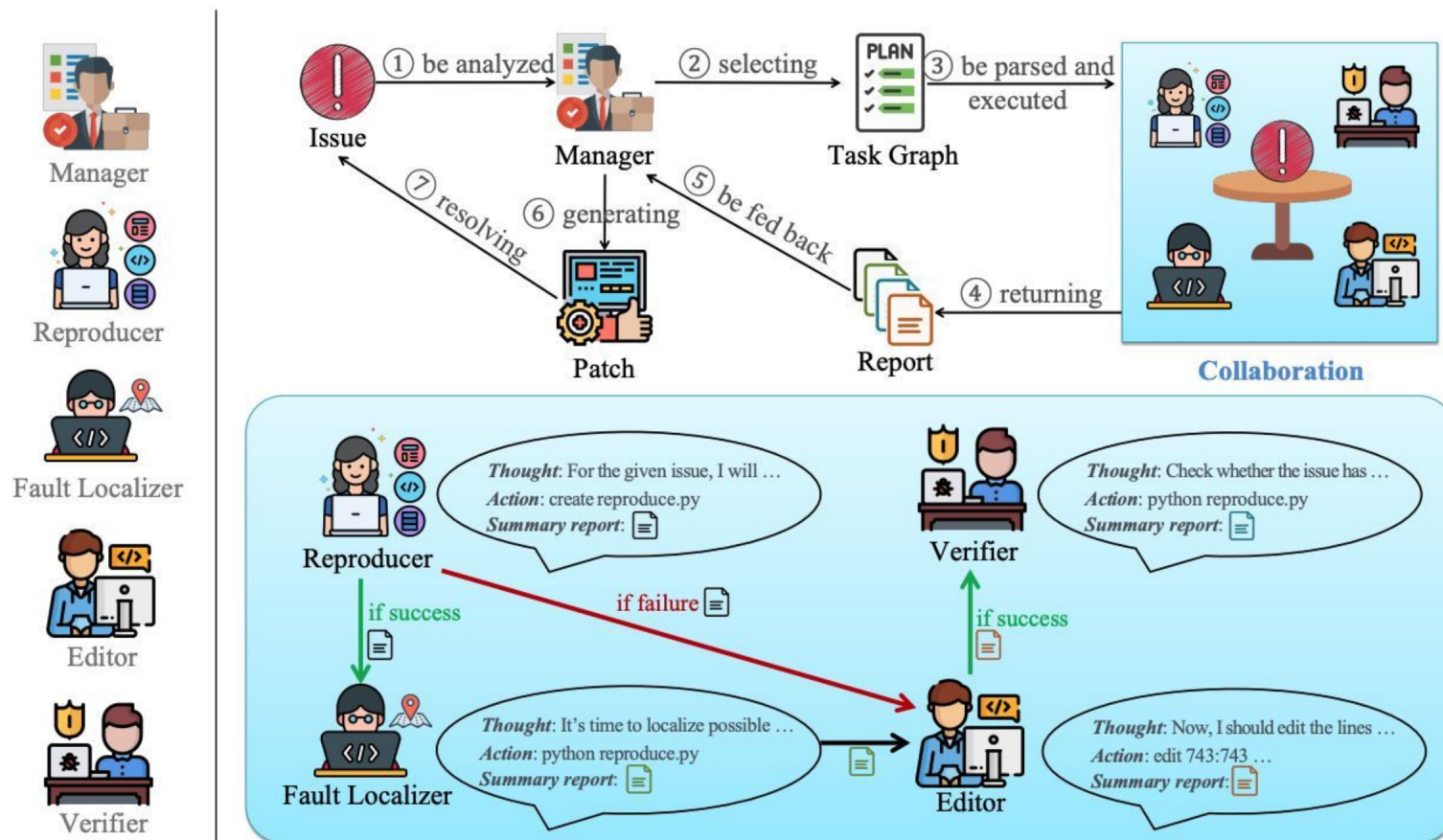
12 repairs, 16 losses. With $a = 0.6$, break-even damage is $b = 0.15$.

A controlled test of communication

Fixed for both conditions	What differs
Initial candidates, tools, final selector	Peer messages versus independent revision
Primary budget and stopping rules	Actual tokens, calls, and latency are measured
Held-out tasks and correctness criterion	Repair, damage, and final success

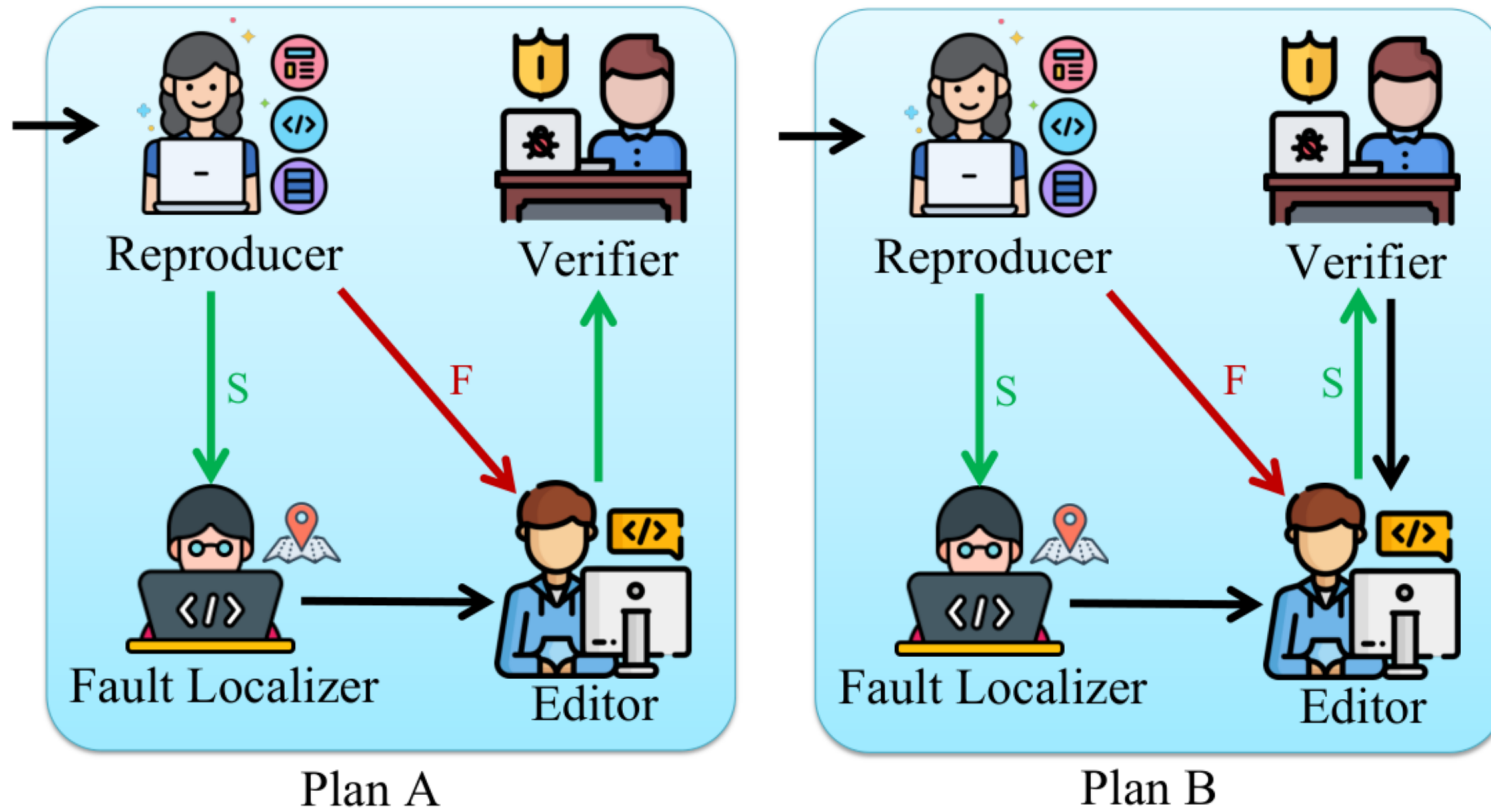
Branch from the same starting candidates; repeat across tasks and random seeds.

CodeR assigns repository repair tasks



Diyi Yang, EMNLP 2024 tutorial, p. 8 (printed slide 155); Chen et al., CodeR, 2024.

CodeR executes a task graph



A: one pass through the roles.

B: test feedback can send work back.

Routing follows success and failure reports.

Specialists give the editor different evidence

Stage	Useful output
Reproducer	A runnable failure case
Fault localizer	Coverage/retrieval ranking
Editor	A candidate patch
Verifier	Observed test outcomes

Coverage requires a runnable reproducer and a successful collection step.

The verifier can edit; official issue-verification tests are excluded from agent access.

Ochiai compares execution with failure

$$O(s) = \frac{\sum_t c_{ts} z_t}{\sqrt{(\sum_t c_{ts})(\sum_t z_t)}} \\ = \frac{e_f(s)}{\sqrt{(e_f(s) + e_p(s))(e_f(s) + n_f(s))}}.$$

$c_{ts} = 1$: test t executes statement s . $z_t = 1$: test t fails.

e_f : failing executions; e_p : passing executions; n_f : failures without execution.

Cosine similarity of binary vectors. No failing tests $\rightarrow$ undefined score.

A ranking from four tests

Constructed example: two failing tests, two passing tests

Candidate	e_f	e_p	n_f	Ochiai
A	2	0	0	1.000
B	2	2	0	0.707
C	1	1	1	0.500

$$O(B) = \frac{2}{\sqrt{4 \cdot 2}} \approx 0.707$$

A can rank first because it lies on the failing path. That alone does not establish causation.

CodeR combines coverage and textual relevance

$$R(F) = \frac{\text{BM25}(F, q)}{\sum_{F' \in \mathcal{F}} \text{BM25}(F', q)}$$

$$S(s) = \lambda O(s) + (1 - \lambda) R(F(s)), \quad \lambda = 0.99.$$

q : issue text. $F(s)$: file containing statement s .

A small retrieval weight can separate statements tied on coverage.

$\lambda = 0.99$ followed a 10-issue exploration; the best weight depends on the metric.

Function localization benefits depend on the cutoff

Top-k precision (%) in the paper's localization evaluation

Setting	Top 1	Top 3	Top 5	Top 10
BM25 only, $\lambda=0$	12.27	25.92	34.04	42.98
Blend, $\lambda=0.99$	16.49	28.33	35.91	43.15
Ochiai only, $\lambda=1$	6.63	14.11	18.23	24.95

Blend – BM25: +4.22 points at top 1; +0.17 points at top 10.

Filtered to 104 issues with usable coverage; this measures ranking, before patch generation.

CodeR's ablation changes quality and computation

Same 50 SWE-bench Lite issues; historical Table 5

Setting	Resolved	Requests	Tokens
CodeR	22% (11/50)	30.40	295K
Without multi-agent	10% (5/50)	18.46	200K
Without localization	14% (7/50)	29.98	309K

The 11-versus-5 result also changes token use: 295K versus 200K.

Average resources per issue. No uncertainty intervals reported; communication is not isolated.

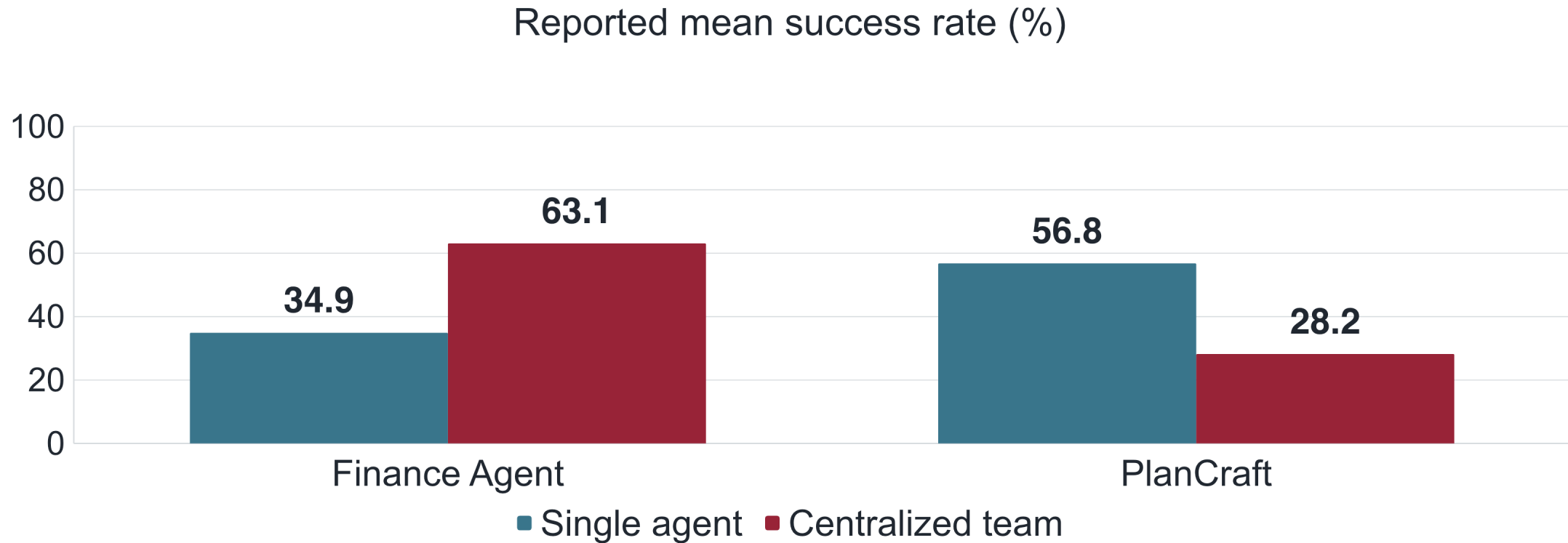
Parallel investigation still has dependencies

Constructed example: investigate a software upgrade failure

Subtask	Inputs	Earliest start
Inspect API changes	Old/new API documentation	Immediately
Inspect dependencies	Lockfiles + release notes	Immediately
Choose a patch	Both investigation results	After both reports
Run tests	Exact patched revision	After edit

Parallel reads can proceed independently; edits and test results depend on repository state.

The same team has different effects across tasks



Finance: +28.2 percentage points. PlanCraft: -28.6 points.

Centralized: 3 workers. Means across tested models; 50 / 100 tasks per configuration.

Compare agent roles from the same initial evidence

Condition	Initial localization evidence	Agent organization
A	Same precomputed ranking + reproducer	One repair agent
B	Same ranking as A	Separate localizer and editor
C	Same ranking as A	Independent repair attempts + fixed selector

Match the primary budget and visible tests; use held-out tests for final scoring.

Report paired success, cost, reproduction failures, and how often the ranking is used.

Choose the team by the failure it can address

Independent attempts: measure whether a correct candidate exists and whether the selector finds it.

Communication: count corrected errors and damaged answers, then compare against independent revision.

Specialization: identify the new observation or tool result, and test whether it improves the final decision.

Appendix: localization depends on usable coverage

300 issues → 140 reproduced → 104 with nonempty coverage

Setting	Top 1	Top 3	Top 5	Top 10
BM25 only, $\lambda=0$	15.32	32.67	42.36	54.07
Blend, $\lambda=0.99$	20.60	36.33	44.44	54.24
Blend, $\lambda=0.01$	23.49	39.21	47.81	56.55
Ochiai only, $\lambda=1$	8.12	16.65	21.19	28.92

File-level precision (%). $\lambda = 0.01$ is best among these displayed rows.

Appendix: predicting which architecture will win

$$E_c = \frac{S}{T/T_{\text{single}}}$$

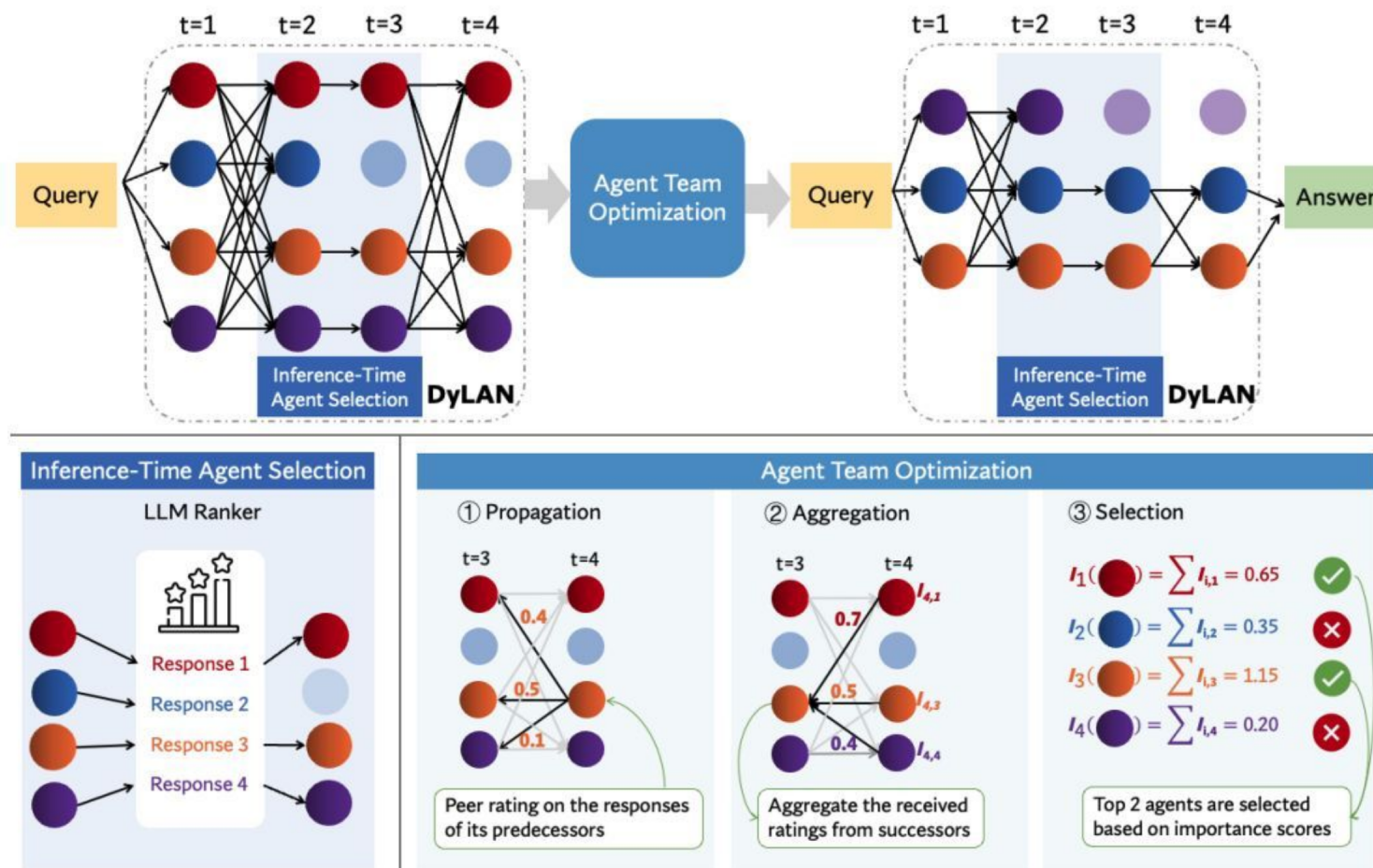
S: success; T: turns. Metrics enter as architecture-level constants.

Were outcome-derived constants estimated within each training fold?

CV $R^2 = 0.373$ (Intelligence Index); 0.413 with the alternative index.

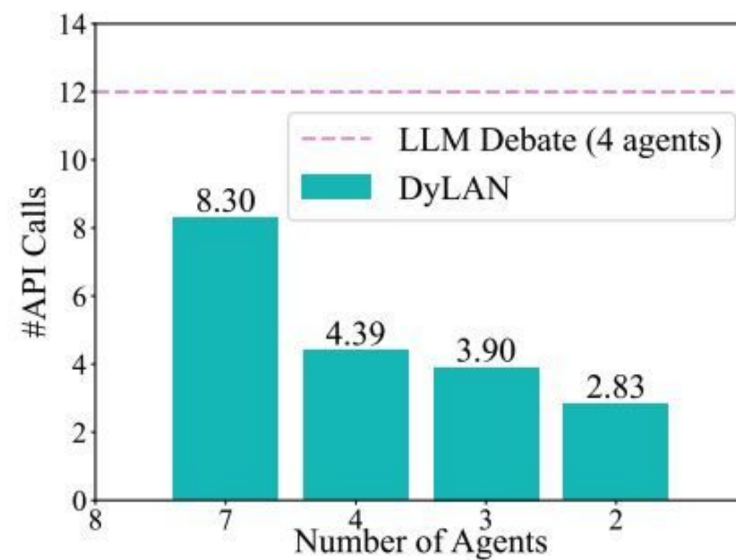
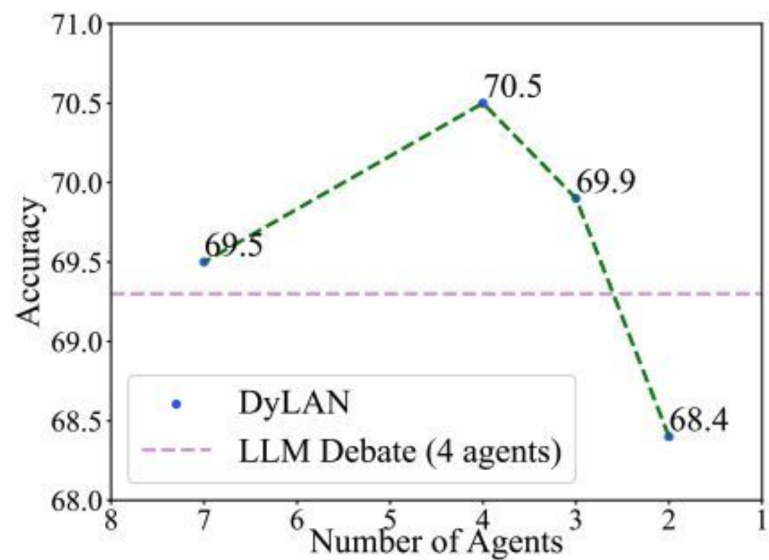
BrowseComp-Plus: Hybrid is recommended for all three held-out models; observed winners differ.

Appendix: DyLAN agent selection



Architecture and selection from Diyi Yang's tutorial and DyLAN (COLM 2024).

Appendix: team size and cost



Historical MMLU/GR setting with the paper's models.

API calls provide a cost proxy, with no direct conversion to tokens or wall time.

The original plot shows no uncertainty bars.

Diyi Yang tutorial p. 24 (printed 171); DyLAN Fig. 3.